Bank AI Calling Platform — Remaining Action Items & Configuration Guide

Prepared for: Platform Administrator & DevOps Lead
Document Date: October 2026
Status: Post-Remediation Manual Action Items

Executive Summary

The engineering review has identified several security, correctness, and concurrency issues that have already been resolved directly in the codebase:
- **SSRF & Twilio Credential Protection**: Fixed unauthenticated webhook vulnerability by enforcing `twilio.validateRequest` and restricting recording URLs strictly to `api.twilio.com`.
- **Call Recording IDOR Fixed**: Restricted recording audio proxying strictly to authenticated Administrators, Supervisors, or the campaign owner.
- **Gemini Model 404 Resolved**: Fixed non-existent `gemini-3.6-flash` model identifier to `gemini-1.5-flash`.
- **Customer Queue Concurrency Lock**: Applied `FOR UPDATE SKIP LOCKED` on database queue selections to prevent duplicate calls to customers.
- **CSV Ingestion Hardened**: Implemented 5MB / 5,000 row limits with 500-record chunked database batching to eliminate out-of-memory and query crashes.
- **Timing Attacks Mitigated**: Replaced vulnerable string comparisons on HMAC signatures and internal secrets with `crypto.timingSafeEqual`.
- **CI/CD Quality Gate**: Upgraded GitHub Actions workflow to run full test suites and TypeScript verification before triggering zero-downtime rolling deploys.

The items detailed below represent the **external infrastructure, provider credentials, and database migrations** that must be configured by you in production.

---

1. Database: Multi-Tenant Billing Schema Migration

Background
Currently, the `BillingAccount` model functions as a shared singleton table without a foreign key relation to individual users or organizations. While functional for single-tenant operations, multi-tenant billing requires isolating credit quotas per account.

Recommended Schema Update
In `bank-ai-calling/prisma/schema.prisma`:
```prisma
model User {
  // ... existing fields ...
  billingAccount BillingAccount?
}

model BillingAccount {
  id                     String             @id @default(cuid())
  userId                 String             @unique
  user                   User               @relation(fields: [userId], references: [id], onDelete: Cascade)
  freeCallsUsed          Int                @default(0)
  freeCallsLimit         Int                @default(10)
  subscriptionStatus     SubscriptionStatus @default(FREE)
  razorpaySubscriptionId String?
  currentPeriodEnd       DateTime?

  usageLogs              CreditUsageLog[]
  updatedAt              DateTime           @updatedAt

  @@map("billing_accounts")
}
```

Action Required
1. Run migration locally:
   ```bash
   cd bank-ai-calling
   npx prisma migrate dev --name link_billing_to_user
   ```
2. In production, run:
   ```bash
   npx prisma migrate deploy
   ```

---

2. GitHub Repository Secrets Setup (For CI/CD)

The updated `.github/workflows/deploy.yml` pipeline runs tests and deploys directly to your EC2 instance. For the pipeline to execute successfully, configure the following secrets in your GitHub repository (**Settings > Secrets and variables > Actions > New repository secret**):

Secret Name	Description	Example / Notes
`EC2_HOST`	Public IP or DNS of your AWS EC2 instance	`54.210.xx.xx` or `ec2-xx.compute.amazonaws.com`
`EC2_USERNAME`	SSH username for the instance	`ubuntu` (for Ubuntu AMI) or `ec2-user`
`EC2_SSH_KEY`	Private SSH Key (`.pem` file contents)	Open your EC2 private key in Notepad, copy full contents including `-----BEGIN RSA PRIVATE KEY-----`

---

3. Production Environment Variables Verification

Ensure the production `.env` files on your EC2 instance (`~/bank-ai-calling-platform/bank-ai-calling/.env` and `~/bank-ai-calling-platform/bank-ai-calling-bridge/.env`) contain all required keys:

Core Security & Inter-Service Secret
- `INTERNAL_API_SECRET`: Must be a cryptographically random string (min 32 characters). Must match exactly between the main application and the bridge server.
  ```bash
  # Generate on Linux/Mac:
  openssl rand -hex 32
  ```
- `NEXTAUTH_SECRET`: Random 32+ character string for signing session JWT tokens.
- `NEXTAUTH_URL`: Canonical public URL of your application (e.g. `https://calling.yourbank.com`).

Twilio Telephony Credentials
- `TWILIO_ACCOUNT_SID`: Master Twilio Account SID (`AC...`).
- `TWILIO_AUTH_TOKEN`: Master Twilio Auth Token.
- `TWILIO_PHONE_NUMBER`: Purchased E.164 phone number (e.g. `+1234567890`).
- `TWILIO_ACCOUNT_SID_PROXY`: (Optional) Read-only subaccount SID if proxying audio.
- `TWILIO_AUTH_TOKEN_PROXY`: (Optional) Read-only subaccount token.

Artificial Intelligence & Voice
- `GOOGLE_GENAI_API_KEY`: Google AI Studio API key with access to `gemini-1.5-flash` and Gemini Live Multimodal WebSocket.
- `AI_AGENT_NAME`: Default agent persona name (e.g. `Priya` or `Sarah`).

Payment Gateway (Razorpay)
- `RAZORPAY_KEY_ID`: Live Razorpay Key ID (`rzp_live_...`).
- `RAZORPAY_KEY_SECRET`: Live Razorpay Key Secret.
- `RAZORPAY_PLAN_ID`: Live recurring subscription plan ID (`plan_...`).
- `RAZORPAY_WEBHOOK_SECRET`: Secret configured in Razorpay Dashboard for webhook signature verification.

---

4. Twilio Webhook Configuration in Twilio Console

Once your domain has an active SSL certificate (HTTPS), configure the Twilio Phone Number in the Twilio Console:
1. Log in to **Twilio Console > Phone Numbers > Manage > Active Numbers**.
2. Click your phone number and navigate to **Voice Configuration**:
   - **A CALL COMES IN**: Webhook, `POST`, `https://<your-bridge-domain>/api/twiml`
   - **STATUS CALLBACK URL**: Webhook, `POST`, `https://<your-bridge-domain>/api/call-status`
   - Select events: `initiated`, `ringing`, `answered`, `completed`.
3. Save changes.

---

5. EC2 Server & Firewall (Security Group) Setup

1. **Inbound Security Group Rules (AWS EC2 Console)**:
   - Port `22` (SSH): Restrict to your office or personal static IP.
   - Port `80` (HTTP): Open to `0.0.0.0/0` (for Let's Encrypt SSL issuance & redirects).
   - Port `443` (HTTPS): Open to `0.0.0.0/0` (public web access).
   - Port `3000` & `8080`: Keep closed from the public internet. All traffic should route through NGINX on ports 80/443.
2. **NGINX Reverse Proxy & SSL Setup**:
   Ensure WebSocket upgrade headers are passed to the bridge container:
   ```nginx
   location /media-stream {
       proxy_pass http://localhost:8080;
       proxy_http_version 1.1;
       proxy_set_header Upgrade $http_upgrade;
       proxy_set_header Connection "Upgrade";
       proxy_set_header Host $host;
       proxy_read_timeout 3600s;
   }
   ```